Module 3, Lecture 3 — Production-Level API Contract Design

Course: Staff Engineer / Technical Lead Mentorship **Module:** 3 — System architecture, technical design, API contracts, ERDs, and data flows **Lecture:** 3 of Module 3 **Case study:** "Spendwise," the personal finance app from Modules 2 and 3 (React Native + FastAPI + PostgreSQL) **Format:** A one-way lecture. Everything is demonstrated for you, including the exercise, a review of a typical attempt, and a reference solution. Nothing here waits for your reply.

Scope note: Lecture 1 fixed the architecture (modular monolith, stateless FastAPI, one PostgreSQL, client-generated UUIDs, `/v1`, OpenAPI as the contract, RFC 9457 errors). Lecture 2 covered the detailed ERD. This lecture designs the **API contract**: the one boundary between the phone and the server that you cannot change quickly once apps are installed. I refer to the tables only where the contract reaches back into the data design. Where this lecture asks the ERD to change, I say so explicitly.

Part 1 — Learning objective

After this lecture you should be able to:

1. Explain why an API contract is a one-way door on mobile, and separate the parts of it that are cheap to change from the parts that are not.
2. Ask the questions a lead asks before drawing a single endpoint.
3. Model resources, URLs, methods, and status codes so that each choice is justified by a requirement or a failure it prevents.
4. Design request and response shapes: IDs, money, dates, nullability, enums, and write responses that can grow without breaking old clients.
5. Design a consistent error model, pagination, filtering, and concurrency control (retries, idempotency, `If-Match`).
6. Put authentication, authorisation, rate limits, and resource limits into the contract rather than bolting them on.
7. Define a versioning and compatibility policy, and test a proposed change against it.
8. Produce an OpenAPI-backed contract, a contract test plan, and ADRs of the right size.

Part 2 — Real-world scenario

The Spendwise Technical Design Document (v0.1) is approved. Lecture 1 promised an API overview and said "full contract to follow." You are the lead and the only engineer. The next task on your board reads: **"Define the API."**

Here is what most developers do. They open the FastAPI tutorial and start writing route functions: `@app.post("/createExpense")`, `@app.get("/getExpenses")`. Each function returns whatever the database query returned. Three weeks later the mobile screens

depend on those exact shapes. Then a requirement changes (Meera wants budgets, and the save response must now carry alert information), and every screen that touched the save response needs rework.

The API is not a by-product of the backend. It is the **product of the backend**, the thing the mobile app actually consumes. And unlike a database column, you cannot migrate it on a schedule you control, because you cannot update the phones.

What you have to work with, in short:

- **Architecture (Lecture 1):** REST-style JSON over HTTPS, `/v1` prefix, OpenAPI generated by FastAPI, TypeScript types generated for the client, bearer access tokens, refresh-token rotation, problem-details errors with a `request_id`, paginated lists, a minimum-supported-app-version endpoint.
- **Data decisions already made:** `amount_minor` as integer paise, `spent_on` as a calendar `DATE`, client-generated UUID for expenses with idempotent create, soft delete, categories as a fixed list stored as data.
- **Relevant NFRs:** save p95 ⩽ 500 ms at 50 concurrent users; exactly one expense on retry or double tap; each user sees only their own data; old app versions keep working; account deletion within 30 days.
- **Open question:** the sign-in method (PRD Q-3, ADR-005 still *Proposed*). The contract must be designed so that this open question does not leak into the other 90% of the API.
- **Constraints:** one engineer, a couple of months, a small budget, and a client who will test with real phones.

This lecture is the hour you spend before writing the first route.

Part 3 — The lead's thought process

3.1 What an API contract actually is

Working definition:

> An API contract is the complete, checkable promise a server makes to its clients about what requests it accepts, what responses it returns, and what it guarantees in between, including the failures.

Three parts of that definition do real work:

- **Complete.** Not just the happy path. Status codes, error shapes, limits, retry behaviour, and ordering guarantees are all part of the contract. If a client depends on it, it is in the contract whether you wrote it down or not. (This is known as *Hyrum's Law*: with enough users, every observable behaviour gets depended on.)
- **Checkable.** A contract that lives in someone's head or a wiki page drifts. A contract that lives in an OpenAPI document, diffed in CI, cannot drift silently.

- **Guarantees in between.** "Calling this twice has the same effect as once" is not visible in a JSON schema, but it is a core part of the Spendwise contract.

Just as in Lecture 1, separate the levels by cost of change:

Level	Question it answers	Example in Spendwise	Cost to change
Contract shape	What resources and fields exist? What do they mean? What are the guarantees?	`amount_minor` is an integer in paise; `POST /v1/expenses` is idempotent on `id`	**Very high** (old apps in pockets)
Contract conventions	How do errors, pagination, versioning, and dates work everywhere?	Problem-details errors; cursor pagination; ISO dates	**High** (touches every endpoint and client)
Implementation	How the server produces the response	Which ORM; how the query is written	Low

A lead spends the most time on the top row. A beginner reverses it and spends days choosing between `snake_case` and `camelCase` (cheap to decide, expensive only if you flip-flop) and no time on whether a response can grow.

The mobile twist. On the web you deploy a new frontend and every user gets it on the next page load. On mobile, an app version can stay in use for months. Store review takes time, and some users never update. So the server must support multiple client versions *at once, for a long time*. Everything in this lecture is shaped by that single fact. The rule that follows:

> **Contract changes must be additive by default. Removing or changing the meaning of something is a breaking change and requires a new version or a long, deliberate deprecation.**

Who are the contract's customers? Three, and a lead keeps all three in mind:

1. **The mobile app** (today: you).
2. **Future developers** reading the OpenAPI document to understand the system.
3. **Attackers**, who read the same contract and try everything it does *not* explicitly allow.

What an inexperienced developer misses: they design endpoints from the database tables outward ("a table per resource, a CRUD route per table") instead of from the client's tasks and the system's guarantees inward.

What an experienced lead investigates: what the client needs to do, what must be true when things fail, and what the contract will look like in a year when requirements have changed.

DBA connection. A published API is the same kind of object as a published schema that other teams query directly. You would not rename a column that ten applications select from. Treat every response field the same way: it is a column other people's code selects.

3.2 The questions a lead asks before drawing endpoints

Eight groups again. They are a checklist for you, not a meeting script.

Group	Questions	What it protects you from
1. Clients and tasks	Who calls this API? What are the five or six things the user actually does? Which screens need which data together?	Endpoints that mirror tables but not tasks; chatty screens
2. Guarantees	Which calls must be safe to retry? Which must be atomic? What ordering or consistency does the client rely on?	Duplicates, lost updates, "worked on my phone" bugs
3. Data representation	How are money, dates, IDs, and optional values represented? Which fields are server-owned? Which are sensitive?	Rounding bugs, month-boundary bugs, data leaks
4. Failure	What can the client observe when things fail (status, body, headers)? What should it do next?	Clients that cannot tell "retry" from "give up"
5. Change	What is certain to change soon? How will a field be added, renamed, or removed? How do old apps learn they are too old?	Breaking every installed app
6. Load and limits	How big can a request be? How many per minute? How big can a list be?	Unbounded work from one user, accidental or deliberate
7. Security	Who may call this? What can they reference? Which fields must never be settable by the client?	BOLA, mass assignment, enumeration
8. Operations and consumers	How will we see which version is calling? How will we test the contract? Who generates client types?	Silent drift, impossible debugging

A back-of-envelope for the contract. Lecture 1's load estimate (about 2.5 writes/s and 12 reads/s at 1,000 users) told us scale is not the challenge. For the contract, the *shapes* matter more than the rates. Do these three short estimates:

```
Largest list a user will ever page:  ~1,000 expenses/month x 10
chars/field... call it
                                     ~250 bytes/expense in JSON -> ~250 KB
per month, unpaged
Typical summary response:            ~8 categories x ~60 bytes = under 1 KB
Typical save response:               ~400 bytes (expense) + ~300 bytes
(budget status/alerts)
Largest "batch" we might ever accept: 100 items x ~250 bytes = ~25 KB request
```

What this tells you:

1. An *unpaged* month of expenses is only about 250 KB, which is manageable, but a *year* is about 3 MB and a lifetime list is unbounded. **Pagination from day one** is cheap insurance, because adding it later is a breaking change (the response goes from an array to an object).
2. Responses are tiny. No need for field selection, sparse fieldsets, or GraphQL-style flexibility for bandwidth reasons.
3. Request sizes are tiny, so a hard **body size limit** (for example 256 KB) will never bother a real client and blocks a whole class of abuse.

3.3 Alternatives considered: API style

As in Lecture 1, never present a single option. The realistic styles for Spendwise:

- **Option A: Resource-oriented REST over JSON** with a few pragmatic action endpoints (sign-in, sign-out).
- **Option B: GraphQL.** One endpoint; the client asks for exactly the fields it needs.
- **Option C: RPC over HTTP / gRPC.** Operations as named methods with typed messages (gRPC uses Protobuf over HTTP/2).
- **Option D: Backend-for-Frontend (BFF) with screen-shaped endpoints.** Endpoints designed per screen (`GET /v1/screens/home`) returning exactly what that screen needs.

Driver or constraint	A. REST + JSON	B. GraphQL	C. gRPC / RPC	D. Screen-shaped (BFF)
Retry safety (NFR-5)	Natural: HTTP semantics, idempotent create by client ID	Mutations are POSTs; idempotency is custom	Custom	Custom
Fits FastAPI + OpenAPI + generated TS types	**Native**	Needs another library and schema	Different toolchain, weaker RN story	Native
Solo engineer operating it	Simple, debuggable with curl	More moving parts: resolvers, query-cost limits, caching	Tooling and proxy complexity for mobile	Simple
Security surface (BOLA, resource limits)	Per-endpoint, easy to reason about	Arbitrary query shapes: depth/cost limits needed	Per-method	Per-endpoint
Evolution with old apps	Additive fields; `/v1`	Additive fields; deprecation built in	Protobuf field rules are strong	Screen changes break old screens
HTTP caching, logs, rate limits per endpoint	Straightforward	Single endpoint hides per-operation cost	Needs gateway support	Straightforward
Client flexibility	Fixed shapes	**Best**	Fixed	Tuned per screen
Learning goal (FastAPI, HTTP, contracts)	**Strong**	Moderate	Weak	Moderate
Fit at ~1,000 users, ~6 tasks	Ample	Overkill	Overkill	Ample, but couples server to screens

My decision: Option A, REST-style JSON with pragmatic exceptions. Notice the reasons:

1. The retry-safety drivers (NFR-5, and later NFR-B5) are expressed in HTTP semantics: `PUT`, `DELETE`, status codes, `ETag` / `If-Match`. REST uses these natively.
2. The bandwidth and flexibility problems that GraphQL solves do not exist here (responses are under 1 KB to a few KB).
3. FastAPI produces OpenAPI from Pydantic models with zero extra tooling, and the client types follow from it. The contract is checked by tools.
4. One engineer can reason about per-endpoint rate limits and per-endpoint authorisation. A flexible query language multiplies the security surface.

An honest note about Option B. GraphQL is the right call when many different clients need different slices of a large graph and you have a team to operate it. A product with five client teams and a big data model would choose differently. Here it is cost without benefit.

An honest note about Option D. Screen-shaped endpoints are tempting for mobile because they reduce round trips. Resist it as the *foundation*: it couples the server contract to screen layouts that change every sprint, and old apps keep calling old screen endpoints forever. Use the cheap middle path: resource endpoints, plus **one** carefully chosen aggregate where the screen genuinely needs data together (the monthly summary is exactly that).

Revisit if:

- A second client type appears with very different data needs (a web dashboard, a partner API).
- Screens routinely need four or more calls to render, with measured latency impact.
- The data model grows into a graph that clients navigate in unpredictable ways.

Pragmatic exceptions to "pure REST." Purity is not the goal; clarity and safety are. Sign-in, refresh, and sign-out are *actions*, not resources, so `POST /v1/auth/sign-in` is clearer than inventing a "session" resource model. Do not burn time defending REST orthodoxy. Be consistent and write the rule down.

3.4 Resource modelling and URLs

Step 1: List the client's tasks, not the tables.

Task (from PRD)	Needs
Create an account, sign in, stay signed in, sign out	Auth endpoints
Record an expense (the most important task)	Create
Fix a mistake	Read one, update
Delete an expense	Delete
See where money went this month	Monthly summary (aggregate)
Browse expenses	Paginated, filterable list
Choose a category	Category list
Export my data	CSV download
Delete my account	Account deletion
Let the app report usage	Usage events
Know whether the app is too old	Meta endpoint

Step 2: Name resources with nouns, plural for collections.

```
/v1/expenses              collection
/v1/expenses/{id}         one expense
/v1/categories            collection (read-only, fixed list)
/v1/summary               a computed view, not a stored collection
/v1/account               the caller's own account (singular: there is only
"mine")
/v1/events                write-only sink for usage events
/v1/meta                  public, tiny, unauthenticated
```

Rules and why they exist:

Rule	Why
No user ID in the path for the caller's own data. Use `/v1/expenses`, not `/v1/users/{userId}/expenses`.	The user comes from the token. A user ID in the path invites BOLA ("change the ID to see someone else's data") and gives a second source of truth that can disagree with the token.
Verbs live in the HTTP method, not the URL.	`POST /v1/createExpense` duplicates what `POST /v1/expenses` already says and teaches clients nothing consistent.
Action endpoints only when no resource fits, and name them as nouns/phrases (`/auth/sign-in`).	Honest pragmatism; keep the exceptions few and listed.
Lowercase, hyphenated paths; one plural form.	Predictability.
Do not nest deeper than one level.	Deep nesting couples URLs to the data model and makes IDs redundant.
Use the resource's natural key where one exists.	See budgets below.

Natural keys and `PUT`. In Release 1.1, a budget is identified by `(user, category)`, and a user has at most one limit per category. So the URL `/v1/budgets/{category_id}` identifies it with no new ID, and `PUT` ("make it exactly this") is the right verb, because `PUT` is naturally idempotent and "set limit to X" has replace semantics. Compare ADR-003: for expenses we *rejected* `PUT /expenses/{uuid}` as create-or-replace, because a retry with different content would silently overwrite a record. The same verb is right for one resource and wrong for another, and the reason is the semantics we want, not habit.

3.5 Methods, status codes, and the guarantees

Every endpoint must answer: *is it safe? is it idempotent? what does the client do on each failure?* (Definitions per HTTP semantics: **safe** = no state change; **idempotent** = repeating the request has the same effect as sending it once.)

Method	Safe	Idempotent by definition	Spendwise use
`GET`	Yes	Yes	Read
`POST`	No	**No**	Create (made idempotent by client ID), actions
`PUT`	No	Yes	Replace (budgets)
`PATCH`	No	**Not guaranteed**	Partial update (expenses)
`DELETE`	No	Yes	Delete

`POST` is not idempotent by default. That is why ADR-003 exists. The contract turns `POST /v1/expenses` into a retry-safe operation by making `id` required and client-generated.

Status codes: a small, fixed vocabulary. Using forty codes helps no one. These are the ones Spendwise uses, and each has one meaning:

Code	Meaning in Spendwise	Client action
200 OK	Success (read, update, **idempotent replay**, batch processed)	Use the body
201 Created	New resource created; `Location` header points to it	Use the body
202 Accepted	Request accepted; work completes later (account deletion)	Show "pending" state
204 No Content	Success, nothing to return (delete, sign-out, events)	Done
304 Not Modified	Conditional GET matched (`ETag`)	Use the cached copy
400 Bad Request	Malformed request that cannot be parsed	Fix the client; do not retry
401 Unauthorized	Missing, expired, or invalid access token	Refresh once, replay once; else sign in
403 Forbidden	Authenticated but not allowed (for example account pending deletion)	Do not retry
404 Not Found	No such resource **or not yours** (see below)	Do not retry
409 Conflict	State conflict (ID reuse with different content, deleted ID reuse)	Do not retry blindly
412 Precondition Failed	`If-Match` did not match the current version	Re-fetch, reconcile
413 Content Too Large	Body over the limit	Fix the client
422 Unprocessable Content	Well-formed but invalid (validation)	Show field errors; do not retry
426 Upgrade Required	App version below the minimum supported	Show the force-upgrade screen
428 Precondition Required	`If-Match` was required but missing	Fix the client
429 Too Many Requests	Rate limit hit; has `Retry-After`	Back off; honour `Retry-After`

Code	Meaning in Spendwise	Client action
500 Internal Server Error	Unexpected server bug	Retry with backoff if safe; report
503 Service Unavailable	Temporarily unavailable (DB down, overload); may have `Retry-After`	Retry with backoff and jitter

Design decisions hidden in that table:

- `404`**, not** `403`**, for other people's resources.** If `GET /v1/expenses/{id}` returned 403 for someone else's expense and 404 for a missing one, an attacker could enumerate which IDs exist. Return the same `404` for both. (Lecture 1, flow 1(d), applied the same idea to the conflict path.)
- `200` **on idempotent replay,** `201` **on first creation.** The client treats both as success and uses the body. The distinction helps debugging and tests. An optional response header such as `Idempotent-Replay: true` is a convenient (non-standard) signal for logs and support.
- `422` **for validation, with FastAPI's default behaviour.** FastAPI returns 422 for body validation errors and, by default, for malformed JSON too. Accept that rather than fight it, but **replace the default error body** with problem-details (see 3.7).
- `DELETE` **is idempotent: deleting something already deleted still returns** `204`**.** The client cannot tell whether its first delete landed, so the second must not fail. (A delete of an ID that never belonged to the caller returns `404`.)
- `POST /v1/expenses` **on a previously *deleted* ID returns** `409` **with a distinct problem type.** This is a corner Lecture 1 did not spell out: the client retries a create after the user already deleted that expense on another screen. The server must not resurrect it, and the client must be able to recognise "this is gone, stop retrying."
- **Account deletion returns** `202`**, not** `204`**.** The two-phase design in Lecture 1 (flow 4) means the account is *pending deletion*, not gone. The status code tells the truth, and the body says when the purge is due.

3.6 Representations: how data looks on the wire

The representation decisions are the ones you cannot take back. Make them once, write them down, and enforce them in the schema.

Rules for the whole API:

Topic	Rule	Why
Format	JSON only (`application/json`); problems as `application/problem+json`.	One format means no content-negotiation bugs.
Field names	`snake_case` everywhere.	Matches Python and the database; the generated TS types keep the names, so the client never translates. Consistency matters more than the style chosen.
IDs	Strings. Expense ID is a UUID the client generates (ADR-003). Other IDs are opaque to the client.	Clients must not parse or do arithmetic on IDs.
Money	`amount_minor`: an **integer** count of minor units (paise), plus `currency_code`. **Never** a float, never a decimal string.	ADR-002: exactness. JSON numbers are doubles in many clients, but integers below 2^53 are exact, and our maximum (1,000,000,000) is far below that.
Calendar dates	`spent_on`: `"2026-10-14"` (ISO 8601 date, no time, no zone).	ADR-002: it is the user's calendar date.
Timestamps	`created_at`, `updated_at`: RFC 3339 in UTC with `Z`, for example `"2026-10-14T13:02:11Z"`.	Unambiguous; sortable. Server-set only.
Months	`"2026-10"` (`YYYY-MM`) for query parameters.	Matches how the summary is asked.
Optional values	Absent vs `null` have defined meanings: in a **response**, a nullable field is **always present** (possibly `null`). In a **PATCH request**, absent = "leave unchanged", `null` = "clear it".	Prevents "did you forget it or mean empty?" ambiguity.
Enums	Strings, lowercase. **Clients must tolerate unknown values** and fall back gracefully.	Lets the server add values without breaking old apps (the *tolerant reader* rule).

Topic	Rule	Why
Server-owned fields	`id` (for server-created), `user_id`, `created_at`, `updated_at`, `version`, `deleted_at` are never accepted from the client. Only `id` is client-owned, for expenses.	Prevents *mass assignment*. See 3.9.
Not exposed	`user_id`, `deleted_at`, password material, internal flags.	Least data in the contract.

Request strictness vs response tolerance.

- **Requests: strict.** Configure Pydantic request models to **forbid unknown fields** (`extra="forbid"`). A client that sends `"ammount_minor"` gets a clear `422` instead of silently saving nothing. It also stops an attacker from probing for fields like `user_id` or `is_admin`.
- **Responses: tolerant.** The *client* ignores fields it does not know. That is what makes additive changes safe.

Strict in, tolerant out. Say it out loud until it is a habit.

The expense representation:

```json
{
  "id": "0192f5c4-7a2e-7c3b-9f10-5d1e2a3b4c5d",
  "amount_minor": 45000,
  "currency_code": "INR",
  "category_id": 3,
  "spent_on": "2026-10-14",
  "note": "Groceries",
  "created_at": "2026-10-14T13:02:11Z",
  "updated_at": "2026-10-14T13:02:11Z",
  "version": 1
}
```

(`version` is new in this lecture; see optimistic concurrency in 3.8 and the note that it reaches back into the ERD.)

The create request:

```json
```

```
{
  "id": "0192f5c4-7a2e-7c3b-9f10-5d1e2a3b4c5d",
  "amount_minor": 45000,
  "category_id": 3,
  "spent_on": "2026-10-14",
  "note": "Groceries"
}
```

Note what is *absent*: `user_id`, `currency_code` (INR only in v1, and the server defaults it), timestamps, and `version`.

A decision about `spent_on` validation. The server range-checks `spent_on`. The upper bound must allow for time zones: a user in UTC+14 can legitimately be a calendar day ahead of a server whose clock is UTC. So the rule is "not more than one day after the server's current date", and "not before a sane lower bound." The exact numbers belong to the PRD (US-2); the *shape* of the rule belongs here.

Write responses that can grow: the envelope question. The save response will soon carry more than the expense (budget status and alerts in Release 1.1). Two ways to make that safe:

- **Option 1: Bare resource plus additive sibling fields.** The response is the expense object. Later we add `budget_status` and `alerts` at the top level. Old clients ignore them.
- **Option 2: Envelope from day one.** `{ "expense": {...} }`, then add siblings.

Both are additive-safe *if chosen now*. The unsafe path is to return a bare resource and later decide to wrap it, which is a breaking change. I choose **Option 1** with the rule written into ADR-007: *single-resource responses are the resource itself; derived, read-only extras on write responses are additional top-level fields.* The cost is that the top level mixes "the resource" and "side-effect information", which I accept for consistency (the same shape from `POST`, `GET`, and `PATCH`). Batch and list responses are envelopes by nature.

List responses are always envelopes:

```json
{ "items": [ ... ], "next_cursor": "eyJzIjoiMjAyNi0xMC0xMyIsImkiOiIwMTky..." }
```

Starting with a bare array (`[ ... ]`) would make adding `next_cursor` later a breaking change. That is the single most common unforced error in API design.

3.7 Errors: a contract inside the contract

Clients spend half their code handling failure. If every endpoint reports errors differently, the client has to special-case each. Pick one format.

The format: Problem Details (RFC 9457, which obsoletes RFC 7807), served as `application/problem+json`:

```json
{
  "type": "https://api.spendwise.example/problems/validation-error",
  "title": "Request validation failed",
  "status": 422,
  "detail": "One or more fields are invalid.",
  "instance": "/v1/expenses",
  "code": "validation_error",
  "request_id": "req_01JA3K9Q7X2M",
  "errors": [
    { "pointer": "/amount_minor", "code": "too_large", "detail": "Must be at mo
    { "pointer": "/spent_on", "code": "date_in_future", "detail": "Date cannot
  ]
}
```

What each member is for:

Member	Purpose
`type`	A stable URI identifying the *kind* of problem. Clients may branch on it.
`title`	Short human-readable summary of the type; does not change per occurrence.
`status`	Repeats the HTTP status for clients that lose the header.
`detail`	Human-readable explanation of *this* occurrence. **Not for parsing.**
`instance`	Identifies this occurrence (here, the path).
`code`	**Our extension:** a short, stable, machine-readable string. Clients branch on `code`.
`request_id`	Correlation ID; also sent as an `X-Request-Id` response header and written to logs.
`errors[]`	**Our extension:** per-field validation problems with a JSON-pointer `pointer`.

Rules:

1. **Clients branch on `code` (or `type`), never on `detail` or `title` text.** Text is for humans and may be reworded.
2. **Codes are part of the contract.** Adding a code is additive. Renaming one is breaking.

3. **Every error, including 401, 404, 429, and 500, uses this format.** Install an exception handler so that framework defaults (FastAPI's default 422 body, 404 pages from a proxy) are converted. The edge platform's own errors (for example a 502 from the load balancer) may not be, so the client must also handle a non-JSON error body gracefully.
4. **Never leak internals.** No stack traces, SQL, or table names in `detail`. The `request_id` is how *you* find the detail in logs.
5. **Security-sensitive errors are deliberately vague.** Sign-in failure is always `invalid_credentials`, never "no such user" vs "wrong password." Conflicts on another user's ID return the same `404`.

An error catalogue is a deliverable. Keep a table of every `code`, its status, and what the client should do. The catalogue is what makes errors checkable and testable:

`code`	Status	Meaning	Client action
`validation_error`	422	Body or query failed validation	Show field errors
`invalid_credentials`	401	Sign-in failed	Show generic message
`token_expired`	401	Access token expired	Refresh once, replay
`refresh_token_reuse`	401	Old refresh token reused; family revoked	Force sign-in
`not_found`	404	Not found *or not yours*	Stop
`expense_id_conflict`	409	ID exists with different content (or not yours)	Treat as client bug; report
`expense_deleted`	409	ID was used by an expense that is now deleted	Drop from the retry queue
`version_mismatch`	412	`If-Match` stale	Re-fetch, reconcile
`precondition_required`	428	`If-Match` missing	Fix client
`payload_too_large`	413	Body over limit	Fix client
`rate_limited`	429	Too many requests	Honour `Retry-After`
`upgrade_required`	426	App version too old	Force-upgrade screen
`account_pending_deletion`	403	Account deletion requested	Show account-deleted screen
`service_unavailable`	503	Temporary failure	Retry with backoff
`internal_error`	500	Unexpected bug	Show friendly error; retry if safe

3.8 Retries, idempotency, and concurrency in the contract

This is where Lecture 1's flows turn into promises that a client can rely on.

1. Create is idempotent on `id` (ADR-003). The contract promise, in one sentence the client team can rely on:

> Sending the same `POST /v1/expenses` body (same `id`, same content) any number of times results in exactly one expense and the same successful response body.

Define "same content" explicitly so there is no argument later: the comparison covers `amount_minor`, `category_id`, `spent_on`, and `note` (after normalisation such as trimming). Timestamps and `version` are server-owned and excluded.

2. Update uses optimistic concurrency. Two phones, or a retry racing with a later edit, can overwrite each other: the *lost update*. The standard HTTP answer is a **version tag**:

```
GET /v1/expenses/{id}
  -> 200, ETag: "3", body includes "version": 3

PATCH /v1/expenses/{id}
  If-Match: "3"
  { "amount_minor": 50000 }
  -> 200 (new ETag "4")            if the current version is 3
  -> 412 version_mismatch          if someone else changed it
```

Decisions:

- **`PATCH` follows JSON Merge Patch semantics**: only the fields sent are changed; `null` clears an optional field (here only `note`); absent means unchanged.
- **`If-Match` is *required* on `PATCH` from v1.0** (missing returns `428`). Why make it strict now? **Tightening a rule later is a breaking change; loosening it later is not.** Our only client is ours and today is the cheapest day to be strict.
- **A retried `PATCH` poses a puzzle.** The first attempt succeeded (version 3 to 4), the response was lost, the client retries with `If-Match: "3"` and gets `412`. The client cannot tell whether it was its own earlier attempt or someone else's edit. The contract-level rule: *on `412`, re-fetch; if the fetched resource already equals the intended state, treat the operation as done.* This is a client rule, but the server contract must document it.
- **This reaches back into the ERD.** `version INTEGER NOT NULL DEFAULT 1`, incremented on every update, becomes a column on `expenses`. That is an example of the cycle from Lecture 1: contract design exposes data design changes. (An `updated_at`-derived ETag is a possible alternative but timestamp resolution and clock issues make an integer cleaner.)

3. Delete and replace are idempotent by nature. `DELETE` repeated returns `204`. `PUT /v1/budgets/{category_id}` repeated with the same body leaves the same state.

4. Conditional GET for rarely-changing data. `GET /v1/categories` is a fixed list. Return an `ETag` and let the client send `If-None-Match`; a `304` costs almost nothing. Do **not** do this for user financial data: those responses carry `Cache-Control: no-store`, because totals must always reflect the latest state (NFR-4) and because intermediaries and device caches must not retain them.

5. A client retry policy is part of the contract. Write it down; do not leave it to each developer:

Situation	Retry?	How
`GET`, timeouts, `502/503/504`, connection reset	Yes	Exponential backoff with jitter, a small maximum (for example 3 attempts)
`POST /v1/expenses` (same `id`)	Yes, same body, same `id`	Same backoff
`429`	Yes, after `Retry-After`	Honour the header
`401 token_expired`	Once, after a refresh	If refresh fails, sign in
`4xx` otherwise (400, 403, 404, 409, 412, 413, 422, 426)	**No**	Fix or surface the error
`500`	Only if the operation is idempotent	With backoff, small cap

Without this table, a client that retries 422s or forgets backoff turns a small outage into a retry storm (Lecture 1, section 3.6).

6. Timeouts. The client sets a total request timeout (for example 10 s). The server sets a database `statement_timeout` well below that so a slow query becomes a clean `503`, not a hung socket.

3.9 Security in the contract

Security is not only a layer; the contract itself decides what attackers may try. OWASP's *API Security Top 10 (2023 edition)* is the checklist; the ones that bite a project like this are below. (Verify the current edition when you use it.)

Risk	How it appears in an API	Contract-level defence
BOLA (broken object-level authorisation)	`GET /expenses/123` returns another user's expense	No user ID in paths; repository filters by token user; `404` **for not-yours**
Broken authentication	Weak sign-in, replayable tokens	Short-lived access token; rotating refresh tokens; rate limits on sign-in; vague `invalid_credentials`
BOPLA / mass assignment (broken object *property* level authorisation)	Client sends `"user_id": "<victim>"` or `"created_at"` and the server blindly copies it	**Separate request and response models;** request models list *only* client-writable fields; `extra="forbid"`
Excessive data exposure	Response returns the whole database row, including internal columns	**Explicit response models** (`response_model=ExpenseOut`); never return ORM objects directly
Unrestricted resource consumption	Giant bodies, huge lists, unlimited batch sizes, hammering sign-in	Body size limit; max `limit` on lists; max items on batch; per-user and per-IP rate limits; export limits
Unrestricted access to sensitive business flows	Scripted account creation, scripted exports	Rate limits per flow; consider CAPTCHA or email verification at sign-up (a product decision)
Improper inventory management	Old versions and forgotten endpoints stay live and unmonitored	Versioned URL; every endpoint in the OpenAPI document; deprecation headers and a sunset plan
Security misconfiguration	Open CORS, verbose errors, debug docs in production	No wildcard CORS (a mobile client does not need CORS at all); problem-details without internals; OpenAPI UI off or protected in production

Authentication in the contract.

- All endpoints except `GET /v1/meta`, sign-up, sign-in, and refresh require `Authorization: Bearer <access_token>`.
- **Token response** (stable regardless of the sign-in method):

```
json
```

```
{
  "access_token": "…",
  "access_token_expires_in": 900,
  "refresh_token": "…",
  "token_type": "Bearer"
}
```

- **Containing ADR-005.** The sign-in method is undecided. The contract handles that by **isolating the uncertainty**: only the *credential* part of sign-in is provisional. The token response, refresh, sign-out, and every other endpoint are unaffected by whether the method turns out to be password or one-time code. If a one-time-code flow wins, we *add* endpoints (request a code, verify a code). The existing ones stay valid. If a managed identity provider wins, the token response shape may change, which would be an ADR-005 outcome and a plain breaking-change decision, taken knowingly. Mark the credential endpoints **provisional** in the OpenAPI document, so nobody builds screens on a contract that may move.
- **Tokens are never in URLs** (they end up in logs and browser histories). Headers only.

Where the user comes from. Always the verified token (a `sub` claim looked up server-side), never a request field.

Sensitive data in transit and at rest of the response. Financial responses use `Cache-Control: no-store`. Never log `Authorization` headers or request bodies. (Lecture 1, logging hygiene.)

Rate limits are contract. The client must be told:

- Limits are per user (authenticated) and per IP (unauthenticated), with different values per endpoint family. Illustrative starting values only; tune from real traffic: sign-in about 10/min per IP; writes about 60/min per user; batch writes about 10/min per user; export about 5/hour per user.
- On breach: `429` with `Retry-After` (seconds), problem type `rate_limited`.
- The IETF is standardising `RateLimit` response headers; verify the current state before depending on them. `Retry-After` on `429` is the part you can rely on today.

3.10 Pagination, filtering, and ordering

Pagination is mandatory on every list endpoint from day one (Lecture 1, cheap insurance).

Style	How	Weakness
Offset (`?offset=100&limit=50`)	Skip N rows	Items shift when rows are inserted or deleted between pages (duplicates or gaps); deep offsets are slow
Cursor / keyset (`?cursor=…&limit=50`)	"Give me rows after this position"	Slightly more work; cannot jump to page 7

Decision: cursor (keyset) pagination. An expense list is mostly "newest first, scroll down," and users add and delete expenses while scrolling. Keyset pagination remains consistent under that and uses the index (`user_id, spent_on` with `id` as tie-breaker). The cursor is an **opaque** string to the client (internally, base64 of `(spent_on, id)`). The contract says: *treat cursors as opaque; do not construct or parse them.* That lets the implementation change later without breaking clients.

```
GET /v1/expenses?month=2026-10&category_id=3&limit=50
→ { "items": [ ... up to 50 ... ], "next_cursor": "…" }   (next_cursor is
null on the last page)

GET /v1/expenses?month=2026-10&category_id=3&limit=50&cursor=<next_cursor>
```

Contract rules for lists:

Rule	Value
Default `limit`	50
Maximum `limit`	100 (larger values are clamped or rejected with `422`; choose one and document it)
Ordering	`spent_on` descending, then `id` descending (UUIDv7 keeps it stable and time-like). **Ordering is documented and stable.**
Filters	`month=YYYY-MM`, `category_id=<int>`; unknown filters are a `422`, not ignored (ignoring a typo'd filter returns the wrong data silently)
Deleted rows	Never appear
Cursor and filters	A cursor is only valid with the same filters it was issued under

Why not return a total count? A `total` requires a second query, can be expensive on large sets, and is inconsistent under concurrent writes. For Spendwise the summary endpoint gives totals where they matter (and they are computed correctly by SQL). Add counts only where a screen needs them.

The summary is not a list. `GET /v1/summary?month=2026-10` returns a fixed-size, computed view:

```json
{
  "month": "2026-10",
  "currency_code": "INR",
  "total_minor": 1250000,
  "categories": [
    { "category_id": 3, "total_minor": 450000 },
    { "category_id": 5, "total_minor": 800000 }
  ]
}
```

Bounded size (at most the number of categories), no pagination needed, computed on read (Lecture 1, flow 2).

3.11 Versioning and evolution

The rule again: old apps will call this API for months. So a contract change is either *additive* (safe) or *breaking* (needs a plan).

What counts as breaking and what does not:

Change	Breaking?	Why
Add a new endpoint	No	Nobody calls it yet
Add an **optional** request field	No	Old clients simply do not send it
Add a **required** request field	**Yes**	Old clients will fail validation
Add a response field	No *if* clients are tolerant readers	Old clients ignore it
Remove or rename a response field	**Yes**	Old clients read it
Change a field's type, unit, or meaning	**Yes**	Silently wrong is the worst kind
Add a new enum value	Only if clients are not tolerant	Hence the "unknown value" rule
Tighten validation (smaller max, new required rule)	**Yes**	Previously valid requests now fail
Loosen validation	Usually no	Clients that were valid stay valid
Change a status code or error `code`	**Yes**	Clients branch on it
Change ordering or pagination behaviour	**Yes**	Clients rely on it
Add a new error `code`	No *if* clients have a default branch	Hence "unknown codes get the generic handler"
Add an optional response header	No	

Versioning mechanism. URL versioning (`/v1`), as already decided in Lecture 1. Alternatives (a version header, media-type versioning) are cleaner in theory but harder to see in logs, harder to test with curl, and easier to get wrong. For a solo mobile project, visible in the URL wins.

Policy (ADR-008):

1. Within `/v1`: **additive only.**
2. A breaking change means `/v2`, run alongside `/v1`, with `/v1` kept until usage drops below an agreed threshold (measured from the `X-App-Version` header and request logs).

3. Deprecation is announced *in the contract*: `Deprecation` and `Sunset` response headers (`Sunset` is RFC 8594; check the current standard for `Deprecation`), plus an entry in the contract changelog, plus the minimum-version endpoint.
4. **Every request carries `X-App-Version` and `X-Platform`** (for example `1.4.2` and `ios`). Cost: two headers. Benefit: you can see which versions still call you and decide when a version can be retired.

The forced-upgrade path from the very first release. Lecture 1 said to plan it from v1.0 because old apps cannot learn it later. The contract for it:

```
GET /v1/meta   (no auth)
→ 200 {
  "api_version": "1",
  "min_supported_app_version": "1.0.0",
  "latest_app_version": "1.0.0"
}
```

and on any authenticated endpoint, if `X-App-Version` is below the minimum, the server returns `426 Upgrade Required` with `code: upgrade_required`. The client must have a screen for it from release one. Add it once, in the first build, and it exists forever.

Operational endpoints are not part of the contract. `GET /healthz` and similar are for the platform's health checks, live outside `/v1`, are not in the client OpenAPI document, and may change at will.

The evolution test. Before approving any contract, run a thought experiment: *"Meera adds a requirement. Does this contract absorb it additively?"* Section 4.5 does exactly this with Release 1.1 (budgets). A contract that fails that test is not finished.

3.12 Contract-first, code-first, and testing the contract

Code-first vs design-first.

Approach	How	Strength	Weakness
Design-first	Write the OpenAPI document by hand, then implement	Clients and servers can start in parallel; contract reviewed before code	Two things to keep in sync unless generated from the spec
Code-first	Write Pydantic models and routes; FastAPI generates OpenAPI	Impossible for spec and code to disagree; fast for one person	The contract can change by accident in a refactor

For a solo engineer on FastAPI, **code-first with discipline** is the right default: *the generated OpenAPI file is committed to the repository (`docs/api/openapi.json`) and CI fails when*

the generated spec differs from the committed one without an intentional update. That turns accidental contract changes into visible diffs in pull requests. With several teams you would switch to design-first.

Tooling that checks the contract (verify current maintenance and fit before adopting any of these):

Tool	Role
FastAPI/Pydantic	Generates the OpenAPI document from request/response models
`openapi-typescript` (or a similar generator)	Generates the TypeScript types the React Native app imports, so the client cannot silently drift
A breaking-change detector such as `oasdiff`	Compares the new spec with the last released spec in CI and **fails the build on a breaking change**
A property-based API tester such as Schemathesis	Generates many requests from the spec and checks responses against it: finds 500s, schema violations, and unhandled edge cases
Your own contract tests	The guarantees that are not in a schema: idempotent replay, `404` not `403`, `412` behaviour, rate limit headers

A short slice of the contract in code (FastAPI + Pydantic, to show how the rules above become enforceable):

```python
```

```python
from datetime import date, datetime
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field

class ExpenseCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")          # strict in
    id: UUID                                           # client-generated (ADR-
    amount_minor: int = Field(gt=0, le=1_000_000_000)  # paise; PRD bounds
    category_id: int = Field(ge=1)
    spent_on: date                                     # calendar date (ADR-002
    note: str | None = Field(default=None, max_length=200)

class ExpenseOut(BaseModel):                           # explicit, never an ORM
    id: UUID
    amount_minor: int
    currency_code: str
    category_id: int
    spent_on: date
    note: str | None
    created_at: datetime
    updated_at: datetime
    version: int

# in the router
@router.post(
    "/expenses",
    response_model=ExpenseOut,
    status_code=201,
    responses={
        200: {"model": ExpenseOut, "description": "Idempotent replay"},
        409: {"description": "ID conflict or deleted", "content": {"application
        422: {"description": "Validation error", "content": {"application/probl
    },
)
def create_expense(body: ExpenseCreate, response: Response, user=Depends(curren
    result = expenses_service.create(user.id, body)    # transaction lives in t
    if result.replayed:
        response.status_code = 200
    return result.expense
```

Three things to see: the request model lists only client-writable fields (BOPLA defence); the response model decides what leaves the server (excessive-exposure defence); and the error responses are *documented in the contract*, not just implemented.

Part 4 — Practical demonstration: from the TDD to an API contract

4.1 The weak approach

A first draft from an inexperienced developer might look like this:

```
POST   /createExpense      body: { userId, amount, category, date }
GET    /getExpenses?userId=5&page=2
POST   /deleteExpense      body: { id }
POST   /login              returns { success: true, token: "..." }
GET    /monthlyReport      returns { "Food": 4500.5, "Travel": 800 }

Errors: 200 OK with { "success": false, "message": "Something went wrong" }
Notes: The API is RESTful and secure.
```

Look at what is wrong, item by item:

1. **Verbs in the URL**, with an inconsistent mix of `GET`/`POST`. `POST /deleteExpense` hides a delete behind a method that proxies, caches, and retry logic treat as non-idempotent.
2. `userId` **in the body and query.** The client chooses whose data it reads. That is BOLA served on a plate.
3. `amount` **as a float** (`4500.5`). Rounding errors in money, with the exact totals NFR-4 forbids.
4. `date` **with no format or meaning.** A timestamp? Local or UTC? Month boundaries become a lottery.
5. **No client ID, no idempotency.** A retry after a lost response duplicates the expense (NFR-5).
6. `200 OK` **for errors.** Proxies, monitors, and retry logic all believe the call succeeded. `success: false` forces every client to parse every body.
7. `message` **strings for errors.** The client has to match text. A copy-edit breaks the app.
8. **Offset pagination (** `page=2` **), and no maximum page size.** Items shift under the user and one request can ask for everything.
9. **No versioning, no minimum-app-version path.** The first rename strands every installed app.
10. **Category by *name* as a key in the summary**, so adding or renaming a category changes the response's field names. A map keyed by data values cannot grow safely.
11. **No limits, no rate limits, no error catalogue.**
12. **"RESTful and secure"**: adjectives. We banned those in Module 2.

4.2 The stronger approach, step by step

Step 1: List the client tasks and the guarantees (3.4, 3.2 group 2). Eleven tasks; four guarantees: idempotent create, no lost updates, no cross-user access, old apps survive.

Step 2: Fix the conventions once (3.6 to 3.11): JSON, `snake_case`, integer money, ISO dates, envelopes for lists, problem-details errors, cursor pagination, additive-only within `/v1`, strict in / tolerant out.

Step 3: Model resources and pick methods (3.4, 3.5): the catalogue in 4.3.

Step 4: Decide request and response shapes, with separate request and response models and explicit server-owned fields (3.6).

Step 5: Define the error catalogue and retry policy (3.7, 3.8).

Step 6: Security pass (3.9): authentication, authorisation, mass assignment, limits, rate limits.

Step 7: Run the evolution test (4.5): add Release 1.1 and ask whether it fits additively.

Step 8: Record the hard-to-reverse decisions as ADRs (4.6).

Step 9: Generate and commit the OpenAPI document; set up CI contract checks (3.12).

Step 10: Read it as the client developer, and as the attacker. Can the client implement every screen? Can the attacker do anything the contract does not explicitly allow?

4.3 The sample API Contract Document

A realistic, compact contract for the Spendwise MVP. The machine-checkable version is the generated OpenAPI file; this human-readable version is what a reviewer reads first.

markdown

```
# API Contract: Spendwise v1

**Version:** 0.1 (Draft)   **Date:** 2026-10-21   **Author:** Tech Lead
**Status:** Draft — for review   **Implements:** TDD v0.1 §7, PRD v0.1
**Related ADRs:** 003, 005 (proposed), 007, 008
**Machine-readable spec:** docs/api/openapi.json (generated; committed; diffed

## 1. Conventions
| Topic | Rule |
|---|---|
| Base path | /v1 |
| Transport | HTTPS only, JSON (application/json); errors application/problem+j
| Auth | Authorization: Bearer <access_token>, except meta, sign-up, sign-in, r
| Request headers | X-App-Version, X-Platform (required on every request) |
| Response headers | X-Request-Id on every response |
| Names | snake_case |
| Money | amount_minor (integer paise) + currency_code |
| Dates | spent_on = YYYY-MM-DD; month params = YYYY-MM |
| Timestamps | RFC 3339 UTC, server-set |
| Lists | { "items": [...], "next_cursor": string|null }; cursor pagination; de
| Writes | Request models reject unknown fields |
| Reads | Clients ignore unknown fields and unknown enum values |
| Caching | User data: Cache-Control: no-store. Categories: ETag + If-None-Matc
| Changes within /v1 | Additive only (see §8) |

## 2. Endpoint catalogue
| Method & path | Auth | Purpose | Success | Idempotent? | Notable errors |
|---|---|---|---|---|---|
| GET /v1/meta | none | Min/latest app version | 200 | Yes (safe) | |
| POST /v1/account | none | Create account (PROVISIONAL, ADR-005) | 201 | n/a |
| POST /v1/auth/sign-in | none | Credentials -> tokens (PROVISIONAL) | 200 | n/
| POST /v1/auth/refresh | refresh token | Rotate tokens | 200 | No (rotation) |
| POST /v1/auth/sign-out | bearer | Revoke refresh token | 204 | Yes | |
| DELETE /v1/account | bearer | Start account deletion | 202 | Yes | |
| GET /v1/categories | bearer | Fixed category list | 200 / 304 | Yes (safe) |
| POST /v1/expenses | bearer | Create expense (client UUID) | 201 / 200 replay
| GET /v1/expenses | bearer | List (paginated; month, category_id) | 200 | Yes
| GET /v1/expenses/{id} | bearer | One expense; returns ETag | 200 | Yes (safe)
| PATCH /v1/expenses/{id} | bearer | Partial update; If-Match required | 200 |
| DELETE /v1/expenses/{id} | bearer | Soft delete | 204 | Yes | 404 (not yours)
| GET /v1/summary?month= | bearer | Totals per category | 200 | Yes (safe) | 42
| GET /v1/export/expenses.csv | bearer | Stream CSV of own data | 200 | Yes (sa
| POST /v1/events | bearer | Usage events (<=50 per call) | 204 | Yes, per even
```

```
Operational endpoints (GET /healthz) are outside /v1 and not part of this contr

## 3. Key payloads
(Expense, create request, token response, summary, list envelope: see lecture §

## 4. Error model
RFC 9457 problem details + `code`, `request_id`, `errors[]`. Full catalogue: do
Clients branch on `code`; unknown codes get the generic handler.

## 5. Guarantees
- G1: POST /v1/expenses with the same id and same content yields exactly one ex
- G2: PATCH requires If-Match; stale versions return 412 and change nothing.
- G3: A user cannot read, change, or detect another user's data (404 for not-yo
- G4: Totals in /v1/summary equal the sum of that user's undeleted expenses for
- G5: Lists are ordered by spent_on desc, id desc, and never include deleted ro

## 6. Limits
| Limit | Value |
|---|---|
| Request body | 256 KB |
| List page size | 100 max |
| Events per call | 50 |
| note length | 200 characters |
| amount_minor | 1 to 1,000,000,000 |
| spent_on | not before 2000-01-01; not after (server date + 1 day) |
| Rate limits (initial, to tune) | sign-in 10/min/IP; writes 60/min/user; expor

## 7. Client retry policy
(See lecture §3.8 table. Same body and same id on retried creates; never retry

## 8. Compatibility policy
Additive changes only within /v1 (ADR-008). Breaking change => /v2 side by side
Deprecations announced via Deprecation/Sunset headers and the changelog.

## 9. Open questions and risks
- Q-3 sign-in method (blocks ADR-005; credential endpoints PROVISIONAL).
- R1: clients that are not tolerant readers will break on additive changes. Mit
- R2: ETag/version column requires an ERD change (to confirm with Lecture 2's E

## 10. Change log
| Version | Date | Change |
|---|---|---|
| 0.1 | 2026-10-21 | First draft |
```

4.4 A slice of the OpenAPI document

The generated document is the checkable artifact. Here is the shape of one endpoint, hand-trimmed, so you can see what the tools see:

yaml

```yaml
openapi: 3.1.0
info: { title: Spendwise API, version: "1.0.0" }
paths:
  /v1/expenses:
    post:
      operationId: createExpense
      summary: Create an expense (idempotent on id)
      security: [ { bearerAuth: [] } ]
      parameters:
        - { name: X-App-Version, in: header, required: true, schema: { type: st
        - { name: X-Platform,    in: header, required: true, schema: { type: st
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/ExpenseCreate" }
      responses:
        "201": { description: Created, headers: { Location: { schema: { type: s
                 content: { application/json: { schema: { $ref: "#/components/s
        "200": { description: Idempotent replay (same id, same content),
                 content: { application/json: { schema: { $ref: "#/components/s
        "401": { $ref: "#/components/responses/Problem401" }
        "409": { $ref: "#/components/responses/Problem409" }
        "422": { $ref: "#/components/responses/Problem422" }
        "429": { $ref: "#/components/responses/Problem429" }
components:
  schemas:
    ExpenseCreate:
      type: object
      additionalProperties: false
      required: [id, amount_minor, category_id, spent_on]
      properties:
        id:           { type: string, format: uuid }
        amount_minor: { type: integer, minimum: 1, maximum: 1000000000 }
        category_id:  { type: integer, minimum: 1 }
        spent_on:     { type: string, format: date }
        note:         { type: [string, "null"], maxLength: 200 }
```

Notice `additionalProperties: false` on the request schema and the *documented* `200` replay and `409`. A schema-aware tool can now generate the TypeScript type and test the endpoint.

4.5 The evolution test: does the contract survive Release 1.1?

Release 1.1 (budgets and alerts) arrives. Walk through each need and ask "additive or breaking?":

New need	Contract change	Additive?
Set, change, remove a limit per category	New endpoints `GET /v1/budgets`, `PUT /v1/budgets/{category_id}`, `DELETE /v1/budgets/{category_id}`	**Yes** (new endpoints)
Save response must include budget status and alerts	Add top-level `budget_status` and `alerts` to the `POST` / `PATCH` expense response	**Yes**, *because* the response is the resource object and extras are siblings (3.6); old apps ignore them. Had we returned a bare array or no object at all, it would not have been
Summary shows spent, limit, remaining, percent used	Add `limit_minor`, `remaining_minor`, `percent_used` (nullable) to each category entry	**Yes** (new optional response fields, always present, `null` when no budget)
Replayed save must return the same alerts	Behaviour of the existing replay path (`200`) now includes `alerts`	**Yes**, but note it is a *guarantee* change that must be added to the contract and tested (G6)
Alert thresholds might change (80, 100, maybe 90 later)	`threshold` is an integer; clients display the value, not hard-coded names	**Yes**, if the client does not branch on specific numbers
Alerts `kind` enum may grow	Enum documented as extensible; clients tolerate unknown values	**Yes**
Push notifications later	New endpoints to register a device token	**Yes**

Release 1.1's save response, as the contract now promises:

```json
```

```
{
  "id": "0192f5c4-…",
  "amount_minor": 45000,
  "currency_code": "INR",
  "category_id": 3,
  "spent_on": "2026-10-14",
  "note": "Groceries",
  "created_at": "2026-10-14T13:02:11Z",
  "updated_at": "2026-10-14T13:02:11Z",
  "version": 1,
  "budget_status": {
    "category_id": 3,
    "month": "2026-10",
    "limit_minor": 1000000,
    "spent_minor": 850000,
    "remaining_minor": 150000,
    "percent_used": 85,
    "state": "under_limit"
  },
  "alerts": [
    { "id": "…", "category_id": 3, "month": "2026-10", "threshold": 80, "fired_
  ]
}
```


`budget_status` is `null` when the category has no limit, and `alerts` is an empty array when none fired. The fields are **present** in both cases (the rule from 3.6). The `state` enum (`under_limit`, `over_limit`) follows the tolerant-reader rule.

What the test tells us: the contract absorbed a major feature with **zero breaking changes**. That is the sign of a well-shaped contract. Had we started with a bare list, floats, no envelope, or `200 {success:false}`, Release 1.1 would have been a rewrite of the client.

4.6 The Architecture Decision Records

Two ADRs, written in full. ADR-003 (idempotent create) and ADR-005 (authentication) from Lecture 1 remain in force.

markdown

```
# ADR-007: Contract conventions for the Spendwise API (REST + JSON, problem det

**Status:** Accepted    **Date:** 2026-10-21    **Decided by:** Tech Lead

## Context
The mobile client will be installed on user phones and cannot be updated on dem
(TDD §9, ASR-7). The API must be retry-safe (NFR-5), expose no cross-user data
and keep totals consistent (NFR-4). One engineer; FastAPI generates OpenAPI.
Conventions are expensive to change once clients ship.

## Options considered
A. REST-style JSON with pragmatic action endpoints — chosen.
B. GraphQL — viable; rejected: solves flexibility/bandwidth problems we do not
   idempotency, cost limiting and per-operation rate limiting become custom wor
C. gRPC/RPC — rejected: weaker React Native fit, extra toolchain, no benefit at
D. Screen-shaped (BFF) endpoints as the foundation — rejected: couples server t
   screen layouts; old screens keep calling old endpoints.
Sub-decisions: list responses are envelopes; offset pagination rejected in favo
cursor (items shift while users add/delete); bare-resource write responses with
additive sibling fields rather than a wrapping envelope; errors as RFC 9457 + `
404 (not 403) for other users' resources; strict request models (extra=forbid),
tolerant client readers.

## Decision
REST-style JSON over HTTPS under /v1. snake_case; money as integer minor units;
as ISO strings; RFC 3339 UTC timestamps. Lists: {items, next_cursor}, default 5
Errors: application/problem+json with stable `code`, `request_id`, and `errors[
Single-resource responses are the resource itself; derived extras are added as
top-level fields. Strict in, tolerant out.

## Consequences
+ Retry safety and ownership checks map directly onto HTTP semantics.
+ Generated OpenAPI and TypeScript types make drift detectable.
+ Release 1.1 (budgets, alerts) can be added without breaking changes (see lect
- Top-level mixing of resource fields and side-effect fields on write responses
- Cursor pagination cannot jump to arbitrary pages; no total counts.
- Clients carry discipline obligations (ignore unknown fields/enums/codes).

## Revisit if
A second client type with very different data needs appears; screens need many
round trips with measured latency impact; the data model becomes a graph client
navigate unpredictably.
```

markdown

```
# ADR-008: Versioning and compatibility policy

**Status:** Accepted    **Date:** 2026-10-21

## Context
Old app versions stay installed for months. Store review delays fixes (R2).
We cannot force-update an app that does not know how to be force-updated.

## Options considered
A. URL versioning (/v1), additive-only within a version — chosen.
B. Version in a request header — rejected: harder to see in logs and to test.
C. Media-type versioning (Accept: application/vnd...+json) — rejected: more
   ceremony, easy to get wrong for a solo team.
D. No versioning; "we will just keep compatible" — rejected: the first unavoida
   breaking change has nowhere to go.

## Decision
Additive changes only within /v1. A breaking change creates /v2, served side by
Every request carries X-App-Version and X-Platform. GET /v1/meta reports the
minimum supported version; authenticated calls below it return 426 upgrade_requ
Deprecation is signalled with Deprecation/Sunset headers and the contract chang
CI compares the generated OpenAPI spec to the last release and fails on breakin

## Consequences
+ Visible, testable compatibility rules; we know which versions still call us.
+ Forced-upgrade path exists from v1.0.
- Running two versions costs effort when /v2 happens; the policy forces deliber
- Two extra headers on every request; some clients may omit them (the server tr
  missing as "unknown version" and logs it rather than rejecting, until v1.1).

## Revisit if
Several client types appear; a partner integrates; usage of an old version cann
be retired for contractual reasons.
```

What to notice about these documents:

1. Each ADR is under a page.
2. Alternatives are real, including viable ones that were not chosen.
3. Consequences list the negatives.
4. ADR-005 is still *Proposed*, and the contract was designed so that its uncertainty is **contained** to two provisional endpoints.

5. The contract document links to ADRs instead of repeating them, and every guarantee (G1 to G5) traces back to a requirement.

Part 5 — Documentation: what the lead records, and how much

Document	Purpose	Detail level	Update rule
OpenAPI document	The checkable contract, source for generated client types and tests	Generated; committed; every endpoint, schema, and documented error response	Regenerate on every change; CI diffs it
API contract document (human)	The reader's overview: conventions, endpoint catalogue, guarantees, limits	4 to 6 pages for an MVP	Update with the spec; keep a change log
Error catalogue	Every `code`, status, and client action	One table	Add rows, never rename
Compatibility policy (ADR-008)	What counts as breaking and how we deprecate	Under one page	Supersede, never rewrite
Contract changelog	What changed per release and when it is safe to remove old behaviour	One line per change, with dates	Every release
Client integration notes	Retry policy, token refresh, tolerant-reader rules, forced-upgrade handling	One page	Update after incidents

Where to keep them. In the repository under `docs/api/`, versioned with the code. A contract that lives in Postman collections on someone's laptop dies there.

Practices that keep a contract healthy:

- **Write examples for every payload.** Examples find mistakes that schemas do not (a field that is technically valid but absurd).
- **Trace guarantees to tests.** G1 to G5 each have an automated test. If a guarantee has no test, it is a wish.
- **Review the contract before building screens on it.** A contract review is far cheaper than a client rewrite.
- **Separate fact from preference.** "RFC 9457 defines `application/problem+json`" is fact. "I use snake_case" is a convention with a reason.

Anti-patterns:

- **Documentation-only contracts**: a wiki that disagrees with the code.
- **Contracts that are "whatever the server returns today."** Nobody knows what is promised.
- **Undocumented behaviour clients depend on** (Hyrum's Law).
- **Endpoints added "just in case."** Each one is a permanent liability and attack surface.
- **A changelog nobody writes.** The day you need to retire v1, you will not know who calls it.

Part 6 — Trade-offs and failure scenarios

Items are labelled **[Established]** where the practice is widely agreed and **[Context]** where your judgment decides.

6.1 `200 OK` for everything [Established]

Scenario. The API returns `200` with `{"success": false}` on errors. The uptime monitor sees 100% success during an outage. The mobile retry library treats `200` as done and drops the user's expense. Support cannot grep logs for failures. **Fix:** real status codes, one error format, and `request_id` on every error.

6.2 Bare-array list responses [Established]

Scenario. `GET /expenses` returns `[...]`. A year later the list is slow, so you want pagination, but `next_cursor` has nowhere to live. You either break every client or invent a header contract nobody reads. **Fix:** envelopes from day one.

6.3 Floats for money [Established]

Scenario. `4500.1 + 0.2` gives `4500.299999999999` in some client code paths. Totals no longer match statements (NFR-4). **Fix:** integer minor units end to end (ADR-002).

6.4 The client chooses the user [Established]

Scenario. The API accepts `userId` in the query or body. A script iterates IDs. **Fix:** the user comes only from the verified token; the path and body never carry it.

6.5 Mass assignment [Established]

Scenario. The server copies the request body into the ORM model. A request includes `"user_id"` or `"version"` or `"deleted_at"`, and the server obliges. **Fix:** separate request models listing only writable fields, `extra="forbid"`, and explicit response models.

6.6 Tightening a rule after release [Established]

Scenario. You launch with an optional `If-Match`, later decide it should be required. Every old app that never sent it now fails with `428`. **Fix:** be strict on day one where you will want to be strict forever; loosening later is free, tightening is breaking.

6.7 Clients that are not tolerant readers [Context]

Scenario. The app crashes on an unknown enum value (`state: "approaching_limit"`) after a server-side addition. The server did nothing "wrong"; the *client* broke the additive-change assumption. **Fix:** tolerant reader rules in the client integration notes, a lint rule or unit test that feeds responses with extra fields and unknown enum values.

6.8 Over-flexible query parameters [Context]

Scenario. A generic `GET /expenses?filter=...&sort=...` mini-language is added "for flexibility." It becomes an injection and performance risk, and a permanent contract. **Fix:** a short whitelist of named filters, unknown parameters rejected with `422`.

6.9 Offset pagination under concurrent writes [Established]

Scenario. A user scrolls the list while the sync adds an expense; page 2 starts with the last item of page 1. **Fix:** keyset pagination with a stable total order.

6.10 Retry storms from a naive client [Established]

Scenario. An outage starts; the client retries every failure immediately, including `422`s, and the load multiplies. **Fix:** the retry policy table, backoff with jitter, no retry on 4xx, and honouring `Retry-After`.

6.11 GraphQL "because it's flexible" [Context]

GraphQL is excellent for the right problem. For six tasks and tiny payloads, the cost (query cost limits, per-operation rate limiting, caching, custom idempotency) exceeds the benefit. Choose it knowingly.

6.12 Over-designing for partners that do not exist [Context]

Lecture 1's rule applies: cheap insurance, not speculative construction. `/v1`, envelopes, cursors, and `X-App-Version` are insurance. A public developer portal, OAuth scopes, API keys, webhooks, and a rate-limit tier system are speculation. Add them when a partner exists.

6.13 The 404 vs 403 debate [Context]

Returning `404` for not-yours hides existence and prevents enumeration, but makes debugging slightly harder (a "not found" that is really an ownership failure). Log the true reason server-side (with the `request_id`) and keep the client response uniform.

6.14 One big "do everything" endpoint [Context]

Scenario. `POST /v1/sync` accepts a bundle of creates, updates, and deletes "to reduce round trips." It works until one operation fails and you must invent partial-failure semantics, ordering, and conflict rules. That is the correct design *for a local-first sync*, which is excluded in v1 (Lecture 1, Option E). Do not smuggle it in. Section 8B shows the one place a batch *is* justified and how bounded it is.

Part 7 — Hands-on exercise

You are the lead. This exercise continues the Spendwise release train: **Release 1.2: Import expenses from the user's own CSV file.**

Meera's reasoning: pilot users already have spreadsheets of last year's spending. If they have to type 300 rows by hand, they will not switch.

Inputs (from the Module 2 reference solution for Release 1.2)

- **FR-I1** The user can import up to **500** expense rows from a CSV file stored on their phone.
- **FR-I2** The app parses the file **on the device**, lets the user map columns to date, amount (rupees), category, and note, and converts to the API's representation (`amount_minor`, `spent_on`, `category_id`).
- **FR-I3** Valid rows are saved. Invalid rows are reported with their **row number and reason**, so the user can fix and re-submit them.
- **FR-I4** Import must be **safe to retry**: no duplicates, no lost rows, even after a lost response.
- **FR-I5** Imported expenses count in monthly totals and budget status (NFR-B1). **Imported rows do not fire budget alerts** (PRD decision: 300 alerts would be noise); the user sees updated budget status when the import ends.
- **NFR-I1** Importing 500 rows completes in **p95 ⩽ 5 s** on a typical connection.
- **NFR-I2** An import must not degrade other users: **save p95 stays ⩽ 500 ms** at 50 concurrent users while an import runs.
- **NFR-I3** No single request may consume unbounded server resources.
- **Excluded:** server-side file parsing; background/asynchronous processing; undo of a whole import; fuzzy duplicate detection against manually entered expenses; any format other than CSV; bank import.

Tasks (about 90 minutes)

1. **Contract drivers.** List the contract-significant requirements for this release (the ASR-equivalents) and say which existing ADRs they touch.
2. **Questions.** Write eight questions, grouped by the eight categories in 3.2. Mark the two that most affect the contract.
3. **Alternatives.** Describe at least three contract shapes for import (including one that is wrong and say why), compare them against the drivers in a table, and choose one.
4. **Contract.** Specify the endpoint(s): method, path, request schema, response schema, status codes, and limits. Include a full JSON example of a request and a response for a batch in which one row is valid, one is a replay, and one is invalid.
5. **Flow and failures.** Write the step-by-step server flow, including the transaction, and for each step what can fail and what the client sees.

6. **Retries and concurrency.** Describe what happens if (a) the client retries a batch after a lost response; (b) the user deletes one imported expense on another screen and the client then retries the batch; (c) two imports for the same user run at once.
7. **Error and security table.** At least six error or failure behaviours and two security risks specific to import, each with a design response.
8. **Evolution test.** Show how the contract absorbs two plausible future changes additively (for example "import notes longer than 200 characters" and "push a progress notification").
9. **Scale triggers.** Three measurable signals and the action each triggers.
10. **ADR.** Write ADR-009 for your main decision, with alternatives, consequences (including negatives), and a revisit-if.

What I'll look for

- Did you notice this is a **bounded, retry-safe write-path** problem, not a file-parsing problem?
- Did you reason about **partial failure** explicitly (what does the client do with a mixed result)?
- Did you protect the server with **limits**, and justify the numbers?
- Is every decision tied to a requirement and a failure it prevents?
- Did you avoid speculative machinery (job queues, workers, new services, server-side parsers)?
- Does the ADR contain a real alternative, honest negatives, and a revisit condition?

Part 8 — Review of a typical attempt

As in Lecture 1, here is the kind of answer a learner at your level commonly writes, and a review that gives hints before corrections.

8.1 A typical first attempt (call the learner "Sam")

> **Alternatives:** (1) `POST /import` with the CSV file as multipart upload; the server reads the file. (2) The app calls `POST /expenses` 500 times. (3) A background job. **Choice:** Upload the file, because it's simplest. **Endpoint:** `POST /import-expenses` — send the file and `userId`. **Response:** `200 { "success": true }` when done, `200 { "success": false, "message": "Error in row 17" }` if anything is wrong, and nothing is saved in that case. **Limits:** none needed, users won't upload huge files. **Retries:** If it fails, the user presses import again. **Security:** Check the file is a .csv. **ADR:** Not needed.

8.2 The review

What Sam did well.

- Sam named three genuinely different shapes, including the "just loop the existing endpoint" option, which is a legitimate baseline.
- Sam noticed that invalid rows need to be reported.
- Sam thought about file validation, which is the right instinct (just aimed at the wrong place).

Gap 1: the file goes to the server. FR-I2 says the app parses on the device, and for good reason. *Hint: what does the server have to defend against if it receives arbitrary files?* **Correction:** server-side parsing means untrusted-file handling (encodings, huge or malformed files, decompression tricks, CSV edge cases) on the critical path, plus a large multipart body. Parsing on the phone keeps the server contract small and typed: JSON rows that go through the same Pydantic validation as a normal save.

Gap 2: `userId` in the body. *Hint: which rule from section 3.4 does this break?* **Correction:** the user is the token. This is BOLA, again.

Gap 3: `200 { success: false }`. *Hint: what does the retry library, the monitor, and the log search see?* **Correction:** real status codes for request-level failures and a **per-row** result structure for row-level outcomes.

Gap 4: all-or-nothing on any bad row. *Hint: FR-I3 says valid rows are saved.* **Correction:** valid rows saved, invalid rows reported with row numbers. All-or-nothing makes the user fix 300 rows before saving 299 good ones.

Gap 5: retries are "press the button again." *Hint: what does pressing again do to rows that were already saved?* **Correction:** without client-generated IDs per row (ADR-003 reused), the second press creates **duplicates** of every row that already landed. NFR-I and FR-I4 are violated.

Gap 6: "no limits needed." *Hint: NFR-I3.* **Correction:** limits are part of the contract: maximum rows per request, maximum body size, rate limits. "Users won't" is not a control; scripts will.

Gap 7: 500 sequential `POST /expenses` calls (alternative 2) dismissed without analysis. *Hint: estimate it.* At about 150 ms per round trip, 500 sequential calls take about 75 s, far beyond NFR-I1. The writes rate limit (about 60/min) would also block it. And without special handling, each save would run the alert check, producing noise FR-I5 forbids.

Gap 8: "check it's a .csv" as the whole security story. The extension is a user-supplied label. The real risks are elsewhere (see the reference solution: oversize requests, mass assignment, and **formula injection** when imported notes are later exported as CSV).

Gap 9: no ADR. A hard-to-reverse contract decision with real alternatives is exactly what an ADR is for.

Overall verdict. Sam reached for the simplest transport (upload a file) and left the hard parts unspecified: partial failure, retries, limits, ownership. The fix is the habit from this

lecture: guarantees first, representation second, failures third, evolution fourth.

Part 8B — Reference solution to the exercise

Try to make your own attempt before reading this.

1. Contract drivers

ID	Driver	Source	Touches
C-1	Retry-safe: a retried batch creates no duplicates and loses no rows	FR-I4	ADR-003 (client UUIDs, idempotent create)
C-2	Partial success: valid rows saved, invalid rows reported per row	FR-I3	New ADR-009
C-3	Bounded work per request and per user	NFR-I3, NFR-I2	ADR-007 (limits), rate limits
C-4	Imported rows count in totals and budget status; **no alerts**	FR-I5, NFR-B1	ADR-006 (alerts only on the normal save path)
C-5	Latency: 500 rows in p95 ⩽ 5 s without hurting other users	NFR-I1, NFR-I2	Batch size, transaction size
C-6	Old apps unaffected; the change is additive	ASR-7	ADR-008
C-7	No server-side file parsing; client sends typed rows	FR-I2	ADR-007
C-8	Ownership: only the caller's data	NFR-7	Token-only user identity

2. Questions (grouped; the two that matter most are starred)

1. **Clients and tasks:** ★ What does the user do with a mixed result: retry only the failures, fix and resubmit, or abandon? (It decides the shape of the per-row response.)
2. **Guarantees:** ★ If the response to a chunk is lost, what must a retry return so the client can finish the import? (Per-row `created` / `replayed` status for every ID.)
3. **Data representation:** How does a row report *which* input row failed, given the server never sees the CSV? (An `index` in the request order, and the client maps it to the CSV row number.)
4. **Failure:** What if a row was saved, then deleted by the user, and then the chunk is retried? (Must not resurrect it; return a distinct per-row reason.)

5. **Change:** Will notes ever exceed 200 characters, or will more fields arrive (tags)? (Design rows as the existing expense create schema, so they change together.)
6. **Load and limits:** What is the largest chunk that keeps transaction time and lock time small? (Reasoned below: 100.)
7. **Security:** What happens to a note that begins with `=` or `+` when the user later exports to CSV and opens it in a spreadsheet? (Formula injection; handle at export.)
8. **Operations:** How will we know imports are hurting the database or being abused? (Per-endpoint metrics: batch size, duration, rejected-row ratio; slow-query log.)

3. Alternatives

Alternative	Description	Verdict
W. Upload the CSV, parse on the server	`POST /v1/imports` multipart, synchronous	**Wrong:** moves untrusted-file parsing onto the critical path, large bodies, memory and timeout risk, all-or-nothing temptation, and the format/column mapping UI would have to live on the server
X. Loop the existing `POST /v1/expenses` 500 times	Zero new server code	Viable but poor: about 75 s sequential, collides with write rate limits, no progress or partial-failure story, and every save would run alert evaluation (violates FR-I5)
Y. Batch endpoint with per-row results (chosen)	`POST /v1/expenses/batch`, up to 100 rows per request; client chunks	Meets C-1 to C-8; no new infrastructure
Z. Asynchronous import job	`POST /v1/imports` returns `202` with a job id; poll `GET /v1/imports/{id}`	Viable for much larger files; rejected for 1.2: needs a queue and worker, two failure modes (accepted-but-lost job), and 500 rows fit comfortably in a few synchronous chunks

Driver	W	X	Y	Z
C-1 Retry-safe	✗ (no row IDs)	✓	✓ (per-row UUID)	✓ (job id)
C-2 Partial success	✗ (typically all-or-nothing)	✓	✓	✓
C-3 Bounded work	✗ (large bodies)	✓ per call, ✗ in aggregate	✓ (max 100 rows)	✓ (job limits)
C-4 No alerts	Possible	✗ (each save alerts)	✓ (separate path)	✓
C-5 Latency	✗ (sync, large)	✗ (about 75 s)	✓ (5 chunks)	✓ (async)
C-7 No server parsing	✗	✓	✓	✓
Operational burden	Medium	Low	Low	Higher (queue, worker, monitoring)

Choice: Y. Revisit when imports exceed a few thousand rows or require long-running processing (then Z).

Why 100 rows per request? Back-of-envelope: 100 rows is about 25 KB, and one bulk insert of 100 rows in a single transaction takes on the order of tens of milliseconds, so a chunk's p95 should be well under a second even with the budget-status reads. Five chunks run sequentially in a couple of seconds, within NFR-I1. Smaller chunks give finer progress feedback and shorter transactions; larger chunks reduce round trips. The number is a *starting value to validate by measurement*, which is why it lives in the contract's limits table rather than in someone's head.

4. The contract

Endpoint: `POST /v1/expenses/batch`

Aspect	Specification
Auth	Bearer token (user from token only)
Request body	`{ "items": [ ExpenseCreate, … ] }`, **1 to 100 items**, body ⩽ 256 KB
Item schema	Exactly the `ExpenseCreate` schema (same validation as a single save, so rules cannot diverge)
Item IDs	Client-generated UUID per item; **duplicate IDs inside one request → whole request** `422` (a client bug)
Success status	`200 OK` **whenever the request was processed**, even if some or all items were rejected
Request-level failures	`401`, `413` (too large), `422` (envelope invalid: not JSON, no `items`, 0 or >100 items, duplicate IDs), `429`, `503`
Per-item outcomes	`created`, `replayed`, `rejected` (with a problem-like `error` object)
Alerts	**Not evaluated** on this path (FR-I5)
Rate limit	About 10 requests/min/user (initial; tune)
Guarantee G7	The same request body sent any number of times produces exactly one expense per valid, non-conflicting item, and a per-item result that is consistent with the first successful processing

Why `200` and not `207`? `207 Multi-Status` comes from WebDAV, is not widely understood by HTTP clients and tools, and some treat anything but `200` as unusual. A per-item status inside a `200` body is the pragmatic, widely used choice. (Judgment call: record it.)

Request example (3 rows):

```json
{
  "items": [
    { "id": "0192f6a0-1111-7000-8000-000000000001", "amount_minor": 12000, "cat
    { "id": "0192f6a0-1111-7000-8000-000000000002", "amount_minor": 45000, "cat
    { "id": "0192f6a0-1111-7000-8000-000000000003", "amount_minor": -500,   "cat
  ]
}
```

Suppose row 2 was already saved by an earlier attempt whose response was lost, and row 3 has an invalid amount.

Response example:

json


```
{
  "results": [
    {
      "index": 0,
      "id": "0192f6a0-1111-7000-8000-000000000001",
      "status": "created",
      "expense": { "id": "0192f6a0-1111-7000-8000-000000000001", "amount_minor"
                   "category_id": 2, "spent_on": "2026-09-03", "note": "Auto fa
                   "created_at": "2026-10-14T13:02:11Z", "updated_at": "2026-10
    },
    {
      "index": 1,
      "id": "0192f6a0-1111-7000-8000-000000000002",
      "status": "replayed",
      "expense": { "id": "0192f6a0-1111-7000-8000-000000000002", "amount_minor"
                   "category_id": 3, "spent_on": "2026-09-04", "note": "Groceri
                   "created_at": "2026-10-14T13:01:58Z", "updated_at": "2026-10
    },
    {
      "index": 2,
      "id": "0192f6a0-1111-7000-8000-000000000003",
      "status": "rejected",
      "error": {
        "code": "validation_error",
        "detail": "amount_minor must be greater than 0.",
        "pointer": "/items/2/amount_minor"
      }
    }
  ],
  "summary": { "created": 1, "replayed": 1, "rejected": 1 },
  "budget_status": [
    { "category_id": 2, "month": "2026-09", "limit_minor": null, "spent_minor":
    { "category_id": 3, "month": "2026-09", "limit_minor": 1000000, "spent_mino
  ]
}
```


The `index` lets the client map each result to its CSV row without the server ever seeing the file. `budget_status` lists only the category-months touched by this request (at most

categories times distinct months in the batch, which is small), and is an additive reuse of the 1.1 shape.

5. Flow and failures

Server flow:

1. Edge receives the request over TLS; enforces the body-size limit.
2. API authenticates the token, identifies the user, checks `X-App-Version`.
3. Rate limit check (batch bucket).
4. Parse the envelope: `items` present, 1 to 100 entries, **IDs unique** within the request. Envelope failures → `422` / `413`, nothing written.
5. Validate **each item independently** against `ExpenseCreate` (see the implementation note below). Invalid items are marked `rejected` in memory; they never touch the database.
6. `BEGIN` one transaction.
7. Bulk-insert the valid items: `INSERT … ON CONFLICT (id) DO NOTHING RETURNING id`.
8. For valid items that did **not** come back from `RETURNING` (conflicts): read them back by `id`.
   - Same user and same content, not deleted → `replayed`.
   - Same user, soft-deleted → `rejected` with `expense_deleted`.
   - Another user's ID, or same ID with different content → `rejected` with `expense_id_conflict` (generic, reveals nothing).
9. Compute budget status for each touched `(category, month)` with one indexed aggregate each (no budget row lock, since no alert rows are written).
10. `COMMIT`.
11. Return `200` with `results`, `summary`, and `budget_status`.
12. The app updates its progress, caches, and the user's failure list; it emits an `import_completed` usage event (counts only; no content).

Implementation note. If the request model were `items: list[ExpenseCreate]`, FastAPI would reject the **entire** request when any item is invalid, which would defeat C-2. So the router accepts `items: list[dict]`, and the service validates each dict against `ExpenseCreate` and collects per-item errors. The contract document must say this explicitly, because the OpenAPI schema alone would suggest "all items must be valid."

Step	What can fail	What the client sees / what happens
1	Body over 256 KB	`413 payload_too_large`; nothing written
2	Bad or expired token	`401`; client refreshes once and replays the same body
2	App too old	`426`; force-upgrade screen
3	Rate limit	`429` + `Retry-After`; client waits, resends the **same** chunk
4	Not JSON, 0 or >100 items, duplicate IDs	`422`; nothing written; client bug
5	Some rows invalid	Those rows `rejected`; others proceed
6 to 7	DB unavailable or timeout	`503`; **transaction not committed**; client retries the same chunk later
7	Statement timeout on bulk insert	Whole transaction rolled back; `503`; retry is safe
8	Row exists, different content	That row `rejected` (`expense_id_conflict`); others still reported
9	Slow aggregate	Bounded by indexes and `statement_timeout`; `503` if exceeded
10	Crash before commit	Rolled back; the retry starts clean
10 to 11	Commit succeeded, response lost	Retry: every previously inserted row is a conflict → `replayed`; invalid rows still `rejected`; **same result set**
12	App killed mid-import	On restart the app resumes with the same UUIDs (kept in local state): completed chunks come back as `replayed`, remaining chunks proceed

6. Retries and concurrency

(a) Retry after a lost response. The retry carries the same IDs. Rows that were inserted conflict on the primary key and come back as `replayed` with the original expenses; invalid rows are still rejected the same way. The client gets the full picture and finishes. No duplicate, no lost row. This is ADR-003 applied row by row, which is why the batch reuses `ExpenseCreate` unchanged.

(b) User deletes an imported row, then the client retries the chunk. The soft-deleted row conflicts on the ID. The service sees `deleted_at IS NOT NULL` and reports `rejected` / `expense_deleted` for that row. The expense is **not** resurrected. The client's rule:

`expense_deleted` *is final, so drop it from the retry list and do not count it as a failure to show the user.* This is the same corner case as single-create (3.5), now handled per row.

(c) Two imports for the same user at once (two devices or a double tap on the import button). The chunks are independent transactions that insert disjoint or identical IDs. Identical IDs resolve through the primary key: exactly one row each, the other side sees `replayed`. Disjoint IDs both succeed. There is no shared lock to contend for (we deliberately do not lock budget rows on this path), so there is no deadlock risk between imports and normal saves. The client disables the Import button while a run is in flight; the database enforces correctness regardless.

7. Error and security table

Item	Risk	Design response
Failure: oversize or too many rows	Memory/CPU spike	Body ⩽ 256 KB, ⩽ 100 items, `413` / `422` before any DB work
Failure: partial invalid rows	User loses good rows	Per-item results; valid rows saved
Failure: DB timeout mid-batch	Half-written import	Single transaction for valid rows; `503`; retry replays safely
Failure: lost response	Duplicates or lost rows	Per-row client UUIDs; `replayed` status
Failure: deleted-then-retried row	Resurrected expense	`expense_deleted` per row; never resurrect
Failure: budget status stale after import	User sees wrong "remaining"	Status computed from the same transaction's committed data, in the response; summary endpoint is computed on read (G4)
Failure: import running during peak	Latency for other users	Chunk size 100, short transactions, batch rate limit, `statement_timeout`, metrics on batch duration
Failure: client sends alert-triggering rows	Alert storm	Batch path never evaluates alerts (contract guarantee, tested)
Security: mass assignment / ownership	`"user_id"` or `"version"` in an item, or another user's ID	`extra="forbid"` on the item schema; user only from token; conflicts on foreign IDs return the same generic rejection
Security: CSV/formula injection	An imported note such as `=HYPERLINK(...)` is exported later and executed by a spreadsheet	Keep the data as entered; **neutralise at export**: prefix cells beginning with `=`, `+`, `-`, `@`, tab, or CR with a single quote (a documented export rule); test with hostile notes
Security: resource exhaustion by scripting	Many max-size batches	Per-user batch rate limit, per-IP limit, body limit, request logging with `request_id`

Item	Risk	Design response
Security: log leakage	Notes or amounts in logs	Log counts and IDs, not content (Lecture 1, logging hygiene)

8. Evolution test

Plausible change	Absorbed additively?
Notes longer than 200 characters	**Yes:** loosening a limit is non-breaking. Old clients that cap at 200 remain valid. (Raising it requires the same column change on the single-save schema; the batch inherits it because items reuse `ExpenseCreate`.)
A `tags` field on expenses	**Yes:** an optional request field and a response field on both single and batch paths
Progress notification when a large import finishes	**Yes:** add new endpoints (`POST /v1/imports`, `GET /v1/imports/{id}`) for Option Z; the batch endpoint stays valid for small imports
A new per-item status such as `skipped`	**Yes**, *because* the contract says clients must tolerate unknown statuses (treat as "not saved, show a generic row note")

9. Scale triggers

Signal	Action
p95 batch request duration > 2 s for 7 days, or lock/IO waits visible during imports	Check the insert path and indexes (maybe too many indexes slow bulk inserts); reduce the chunk size from 100 to 50
Rejected-row ratio > 30% sustained, or repeated `422`s on the batch endpoint	Review the client's mapping UI and validation messages; this is a product/UX signal, not a server problem
Weekly imports exceed N files above 500 rows, or users ask for much larger files	Introduce Option Z: async import job with a worker, keeping the batch endpoint unchanged
Database CPU > 60% sustained during imports	Scale the instance up before any redesign (Lecture 1, "scale up before out")

10. ADR-009

markdown

```
# ADR-009: Import via a bounded batch endpoint with per-item results

**Status:** Accepted    **Date:** 2026-11-17    **Decided by:** Tech Lead

## Context
Release 1.2 lets users import up to 500 expenses from their own CSV. The app pa
file on the device (FR-I2). The import must be retry-safe (FR-I4), save valid r
reporting invalid ones (FR-I3), avoid alert noise (FR-I5), stay within p95 5 s
and not degrade other users (NFR-I2, NFR-I3). The existing contract guarantees
idempotent single create keyed by a client-generated UUID (ADR-003).

## Options considered
A. Upload the CSV and parse on the server — rejected: untrusted-file handling o
   critical path, large bodies, all-or-nothing temptation, no row IDs for retry
B. Loop the single-expense endpoint from the client — viable baseline; rejected
   at 500 rows, collides with write rate limits, no partial-failure model, runs
   evaluation per row (violates FR-I5).
C. Batch endpoint, <=100 items per request, per-item results — chosen.
D. Asynchronous import job with queue and worker — rejected for 1.2: extra infr
   and failure modes for a workload that fits in a few synchronous chunks; revi

## Decision
Add POST /v1/expenses/batch accepting 1 to 100 items (each exactly ExpenseCreat
Process in one transaction: validate each item independently, bulk-insert valid
INSERT ... ON CONFLICT (id) DO NOTHING, read back conflicts to classify them as
replayed or rejected, compute budget status for touched category-months, commit
Return 200 with per-item results (created | replayed | rejected), a summary, an
budget_status. Alerts are not evaluated on this path. The client chunks 500 row
to five requests and keeps per-row UUIDs so a retry is safe.

## Consequences
+ Retry-safe by construction; reuses the single-create validation and idempoten
+ No new infrastructure; bounded work per request; partial success supported.
+ Alerts stay on the normal save path, so import cannot cause alert storms.
- Two write paths (single and batch) must keep their rules in step; a shared se
  function and shared test cases are required.
- The OpenAPI schema does not by itself express "valid items proceed even if ot
  invalid"; the contract document and tests must state it.
- A request of 100 items holds one transaction a little longer than a single sa
  monitored via the batch duration metric.
- Clients must implement chunking, resume, and the per-row error UI.

## Revisit if
Imports routinely exceed a few thousand rows; processing exceeds the request ti
```

```
imports need to fire alerts or notifications; batch p95 breaches its trigger; o
write types need batching (then design a general pattern rather than copying th
```

What to notice in the reference. The solution introduced no new service, no queue, no cache, and no file parser. It solved a hard problem (bounded, partial-success, retry-safe bulk writes) by reusing an idempotent create, one transaction, and a per-item result envelope. It also surfaced two product questions that go back to Meera: should the user be told about `expense_deleted` rows, and should imports ever trigger alerts? Contract design exposes requirements, just as architecture does.

Part 9 — Recall test, with answers

Cover the answers and try first.

Q1. Why is the API contract harder to change than a database column? *Answer:* Installed mobile apps depend on it, and you cannot update phones on demand. Old versions stay in use for months, so the server must support multiple client versions at once; contract changes must be additive by default.

Q2. Give three examples of breaking changes and three of additive changes. *Answer:* Breaking: removing or renaming a response field; making an optional request field required; tightening validation or changing a status or error code. Additive: adding an endpoint; adding an optional request field; adding a response field (to tolerant clients).

Q3. Why return `404` and not `403` for another user's expense? *Answer:* Distinguishing them lets an attacker enumerate which IDs exist. A uniform `404` reveals nothing. The true reason is logged server-side under the `request_id`.

Q4. Explain how a client-generated UUID, a primary key, and a `200` replay make `POST /v1/expenses` safe to retry. *Answer:* The retry carries the same `id`; the primary key permits only one row; the server reads the existing row back and returns `200` with it when the content matches, so the client sees exactly one expense even if the first response was lost.

Q5. Why must list responses be envelopes, and why cursor pagination? *Answer:* A bare array has nowhere to put `next_cursor`; adding it later is breaking. Cursor (keyset) pagination stays consistent while items are inserted or deleted and uses the index; offset pagination shifts and gets slow.

Q6. What is mass assignment (BOPLA), and what is the structural defence? *Answer:* The server copies client-supplied fields into a record without restricting which ones, letting the client set server-owned fields such as `user_id` or `version`. Defence: separate request and response models, `extra="forbid"` on requests, and server-owned fields never accepted from the body.

Q7. Why make `If-Match` required from v1.0 instead of optional? *Answer:* Tightening a rule later is a breaking change; loosening is not. The only client today is ours, so strictness is cheapest now.

Q8. Why do errors use stable `code` values, and why must clients not parse `detail`? *Answer:* `code` and `type` are part of the contract and safe to branch on. `detail` is human text that may be reworded; parsing it makes every copy-edit a potential breaking change.

Q9. A client asks to add a required field to an existing request. How do you handle it? *Answer:* It is breaking. Make the field optional with a server default for old clients, or introduce `/v2`. Record the decision and add it to the changelog.

Q10 (challenge). The client says: "Let's switch to GraphQL so the app can ask for exactly the fields it needs." How do you respond? *Answer:* Explain that responses are under a few KB, so bandwidth is not the problem; GraphQL would add query-cost limiting, per-operation rate limits, and custom idempotency, increasing the security and operations surface for a solo engineer. Offer targeted aggregates (the summary endpoint) where screens need data together, document the revisit conditions (a second client type, many round trips per screen, graph-shaped data), and record the decision in an ADR.

Part 10 — Completion checklist

Tick honestly. Revisit the section noted for any you can't.

- [ ] I can explain why an API contract is a one-way door on mobile, and what "additive by default" means (3.1, 3.11).
- [ ] I can list the client tasks and guarantees before drawing endpoints (3.2, 3.4).
- [ ] I can compare API styles against drivers and justify REST here, with revisit conditions (3.3).
- [ ] I can model resources and URLs, and explain why no user ID appears in paths (3.4).
- [ ] I can state which methods are safe and idempotent, and choose from a small, fixed status-code vocabulary (3.5).
- [ ] I can define representations for money, dates, IDs, nullability, and enums, and explain strict-in / tolerant-out (3.6).
- [ ] I can explain why list responses are envelopes and why write responses must be extensible (3.6, 4.5).
- [ ] I can design an error format (RFC 9457 plus `code` and `request_id`) and an error catalogue (3.7).
- [ ] I can explain idempotent create, optimistic concurrency with `If-Match`, and the client retry policy (3.8).
- [ ] I can apply the OWASP API risks (BOLA, mass assignment, excessive exposure, resource consumption) to a contract (3.9).
- [ ] I can design cursor pagination and filtering with documented ordering and limits (3.10).

☐ I can write a versioning and compatibility policy and run the evolution test on a new requirement (3.11, 4.5).
☐ I can describe code-first vs design-first, and how CI protects the contract (3.12).
☐ I can write a right-sized API contract document and ADRs with traceability to requirements (4.3, 4.6).
☐ I can design a bounded, retry-safe batch endpoint with partial-success semantics (8B).
☐ I can explain why contract choices are context-dependent and name the conditions that would change mine (3.3, 6).

Closing: what you should understand before moving on

Before the next lecture, make sure you can say these five things in your own words:

1. **The contract is the product of the backend.** It is a one-way door on mobile; design it from client tasks and guarantees, not from tables.
2. **Make the hard-to-change decisions once, in writing:** representations, errors, pagination, versioning. Strict in, tolerant out.
3. **Guarantees are part of the contract.** Idempotent create, `If-Match`, `404` for not-yours, and bounded batches are promises the schema alone cannot show, so they need documentation and tests.
4. **Design failure into the contract:** distinct status codes, stable error codes, a retry policy, and limits, so clients know what to do and attackers find nothing extra.
5. **Run the evolution test.** If a plausible next requirement forces a breaking change, the contract is not finished.

This is the end of Module 3, Lecture 3. I'm stopping here, as instructed.